

Main Page - Goals

CS370

Multiagent Learning & Computational Game Solving, Notes & Planning

Deliverables:

- Assignment 1 (Applied Focus)
 - Due Date: End of February
 - Length: Minimum 1000 words
 - **Requirements:**
 - Paper Selection: Choose a central applied paper within the fields of multiagent learning or computational game solving.
 - **Summary Contents:**
 - Key Findings: Summarize the main results and conclusions of the paper.
 - Techniques Used: Describe the methodologies and approaches employed in the research.
 - Future Work Directions: Identify and discuss potential avenues for future research suggested by the paper.
- Assignment 2 (Theoretical Focus)
 - Due Date: End of March
 - Length: Minimum 1000 words
 - **Requirements:**
 - Paper Selection: Choose a central theoretical paper within the fields of multiagent learning or computational game solving.
 - **Summary Contents:**
 - Key Findings: Summarize the main results and conclusions of the paper.
 - Techniques Used: Describe the methodologies and approaches employed in the research.
 - Future Work Directions: Identify and discuss potential avenues for future research suggested by the paper.
-
- Final Research Project Proposal Writing Assignment
 - *This assignment involves developing a proposal for a research project based on your readings and interests in the field.*
 - **Due Date: end of semester**
 - **Length:** Minimum 2000 words
 - **Requirements:**
 - **Research Direction:**
 - Novelty and Tractability: Identify a research direction that is both novel and feasible to pursue.
 - Based on Readings: the proposal should be grounded in insights gained from your previous readings and assignments

- **Proposal Contents:**
 - Planning note: use this section to organize the final proposal direction.

Assignment 1:

Sandholm's Computational Game Solving

Sandholm's Computational Game Solving

IMPORTANT TERMINOLOGY:

- Agent = player
- Action = move = choice that agent can make at a point in the game
- Strategy s_i = mapping from history (to the extent that the agent i can distinguish) to actions
- Strategy set S_i = strategies available to the agent
- Strategy profile $(s_1, s_2, \dots, s_{|A|})$ = one strategy for each agent

NOTES:

- Agent's utility is determined after each agent (including nature that is used to model uncertainty) has chosen its strategy, and game has been played:
 $u_i = u_i(s_1, s_2, \dots, s_{|A|})$
- The main focus of the course is Multi-step imperfect-information games
 - Multi-step imperfect-information games means
- **Examples of incomplete-information games with sequential & simultaneous moves:**
 - Negotiation
 - Multi-stage auctions (e.g., FCC ascending, combinatorial auctions)
 - Sequential auctions of multiple items
 - A robot facing adversaries in uncertain, stochastic environment

- Card games, e.g., poker
- Currency attacks
- International
- (over-)fishing
- Political campaigns (e.g., TV spending in each region)
- Ownership games (polar regions, moons, planets)
- Allocating and timing troops/armaments to locations
 - e.g. US allocating troops in Afghanistan & Iraq
 - Military spending games e.g. space vs. ocean
 - Airport security, air marshalls, coast guard, rail
 - cybersec

● **Perfect-information games:**

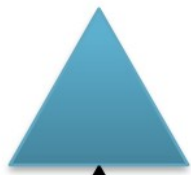
- Checkers
- Chess
- Go
- The techniques we use in perfect-info games don't apply to others because of these issues:
 - Private information
 - concealed cards, like own or opponents
 - In perfect-information games, all players have access to all information at all times. There are no hidden elements; every player's position and possible moves are visible to all participants.
 - Need to understand signals and how other players will interpret signals
 - Definition: Signals in poker refer to any cues or tells that might give insight into a player's hand or intentions. These can be betting patterns, timing, physical gestures, or expressions.
 - Players need to not only be aware of their own signals but also interpret the signals of others to make informed decisions.
 - Must be able to read opponents

- Need to understand deception
 - Understand when / how you or others could bluff
- Need to deceive
 - Bluffing
 - via many avenues like faking nervousness, or showing excessive confidence
- When there is one agent, the maximizing strategy is well-defined. But in a multiagent system, the outcome may depend on others' strategies also the agent's best strategy may depend on what strategies the other agent(s) choose, and vice versa

<https://www.science.org/doi/10.1126/science.aay2400>

Texas Hold'em Rules Reference

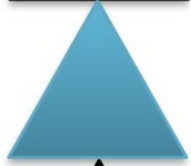
Texas hold'em



Chance deals 2 cards to each player



Round of betting



Chance deals 3 shared cards



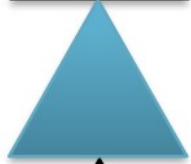
Round of betting



Chance deals 1 shared card



Round of betting



Chance deals 1 shared card



Round of betting

<https://www.youtube.com/watch?v=2dX0lwaQRX0>

Sandholm's Research Paper

Sandholm's Research Paper

equilibrium concepts of von Neumann and Nash have been applied to many real-world challenges such as auctions, cybersecurity, and pricing.

- The fundamental theorem of John von Neumann's game theory states that in a broad category of games it is always possible to find an equilibrium from which neither player should deviate unilaterally
- A Nash equilibrium is a list of strategies, one for each player, in which no player can improve by deviating to a different strategy. (class content, #1 on homework)
 - For example, the Nash equilibrium strategy for Rock-Paper-Scissors is to randomly pick Rock, Paper, or Scissors with equal probability. Against such a strategy, the best that an opponent can do in expectation is tie
 - In this simple case, playing the Nash equilibrium also guarantees that the player will not win in expectation. However, in more complex games, even determining how to tie against a Nash equilibrium may be difficult; if the opponent ever chooses suboptimal actions, then playing the Nash equilibrium will indeed result in victory in expectation.
 - In principle, playing the Nash equilibrium can be combined with opponent exploitation by initially playing the equilibrium strategy and then over time shifting to a strategy that exploits the opponent's observed weaknesses (for example, by switching to always playing Paper against an opponent that always plays Rock)
 - Although a Nash equilibrium strategy is guaranteed to exist in any finite game, efficient algorithms for finding one are only proven to exist for special classes of games, among which two-player zero-sum games are the most prominent. No polynomial-time algorithm is known for finding a Nash equilibrium in two-player non-zero-sum games, and the existence of one would have sweeping surprising implications in computational complexity theory
- a stable state of a system involving the interaction of different participants, in which no participant can gain by a unilateral change of strategy if the strategies of the others remain unchanged.
 - Personal note: Nash equilibrium in game theory is not the same as zugzwang in chess; while both involve a situation where a player is stuck with a strategy due to the actions of others, a Nash equilibrium implies a stable state where no player can improve their outcome by changing their strategy alone, whereas zugzwang means a player is forced to make a move that worsens their position because any move they make will be detrimental.
- **Even if Nash is found:**
 - Moreover, even if a Nash equilibrium could be computed efficiently in a game with more than two players, it is not clear that playing such an equilibrium strategy would be wise. If each player in such a game independently computes and plays a Nash equilibrium, the list of strategies that they play (one strategy per player) may not be a Nash equilibrium and players might have an incentive to deviate to a different strategy.

Two-player game: objective function $f(x,y)$.

First player (x) is trying to minimize, and the second player (y) is trying to maximize.

Suppose $f(x,y) = y^T A x$, where A is some "game matrix", and x plays actions from a set X , y plays actions from a set Y .

We say x, y is a Nash equilibrium if:

$$\max_{\{y' \text{ in } Y\}} f(x, y') = f(x, y) = \min_{\{x' \text{ in } X\}} f(x', y).$$

RPS example:

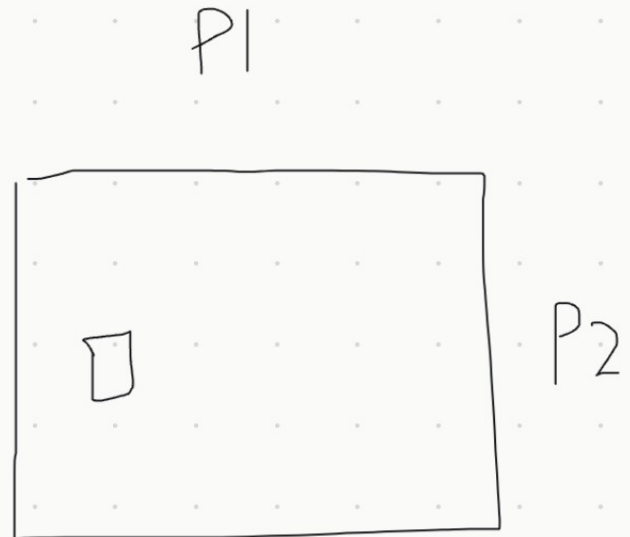
$X = Y =$ probability distributions in R^3 , e.g., $x = (1/2, 1/4, 1/4)$ in X

So let $x = (1/3, 1/3, 1/3)$, $y = (1/3, 1/3, 1/3)$. We claim this is a Nash equilibrium.

Proof: $f(x, y) = 0$

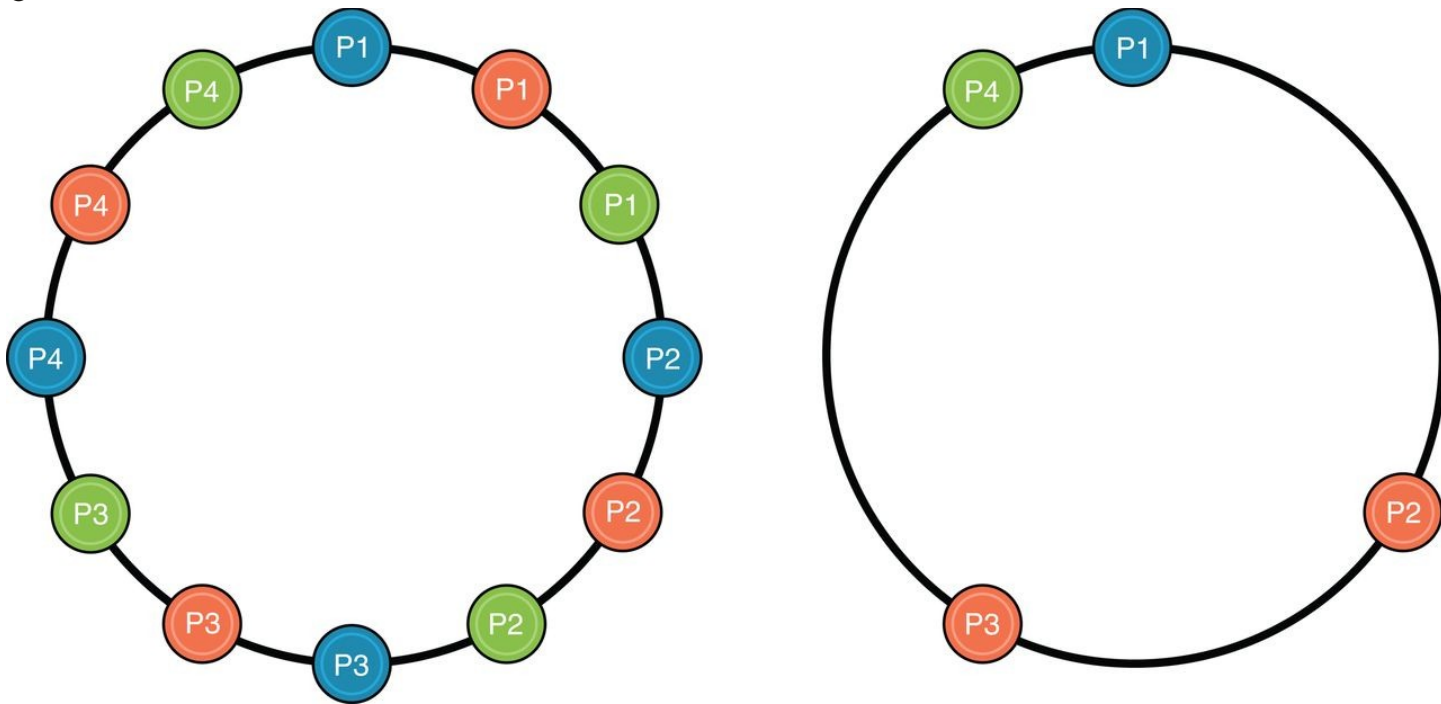
Let y' be any other strategy. I claim that $f(x, y') = 0$

Counterexample: $x = (1/3, 1/3, 1/3)$, $y = (1/2, 1/4, 1/4)$. We claim this is not a Nash equilibrium.



Pluribus:

- plays a fixed strategy that does not adapt to the observed tendencies of the opponents.
- One example of this is the Lemonade Stand Game (17), illustrated in Fig. 1Opens in image viewer, in which each player simultaneously picks a point on a ring and wants to be as far away as possible from any other player.
- The algorithms that we used to construct Pluribus, discussed in the next two sections, are not guaranteed to converge to a Nash equilibrium outside of two-player zero-sum games. Nevertheless, we observe that Pluribus plays a strong strategy in multiplayer poker that is capable of consistently defeating elite human professionals. This shows that even though the techniques do not have known strong theoretical guarantees on performance outside of the two-player zero-sum setting, they are nevertheless capable of producing superhuman strategies in a wider class of strategic settings
-



● HOW WAS THE STRATEGY MADE:

- The core of Pluribus's strategy was computed through self-play, in which the AI plays against copies of itself, without any data of human or prior AI play used as input. The AI starts from scratch by playing randomly and gradually improves as it determines which actions, and which probability distribution over those actions, lead to better outcomes against earlier versions of its strategy. Forms of self-play have previously been used to generate powerful AIs in two-player zero-sum games such as backgammon (18), Go (9, 19), Dota 2 (20), StarCraft 2 (21), and two-player poker (4–6), though the precise algorithms that were used have varied widely. Although it is easy to construct toy games with more than two players in which commonly used self-play algorithms fail to converge to a meaningful solution (22), **in practice self-play has nevertheless been shown to do reasonably well in some games with more than two players**

● HOW IT WORKS:

- **(PHASE 1)** Pluribus's self-play produces a strategy for the entire game offline, which we refer to as the blueprint strategy.
 - Then during actual play against opponents, **(PHASE 2)** Pluribus improves upon the blueprint strategy by searching for a better strategy in real time for the situations in which it finds itself during the game.
- blueprint strategy in Pluribus was computed using a variant of counterfactual regret minimization (CFR)
- CFR is an iterative self-play algorithm in which the AI starts by playing completely at random but gradually improves by learning to beat earlier versions of itself.
 - Every competitive Texas hold'em AI for at least the past 6 years has computed its strategy using some variant of CFR
 - We use a form of Monte Carlo CFR (MCCFR) that samples actions in the game tree rather than traversing the entire game tree on each iteration

● Prices & hardware allocations:

- The blueprint strategy for Pluribus was computed in 8 days on a 64-core server for a total of 12,400 CPU core hours. It required less than 512 GB of memory. At current cloud computing spot instance rates, this would cost about \$144 to produce.
- This is in sharp contrast to all the other recent superhuman AI milestones for games, which used large numbers of servers and/or farms of graphics processing units (GPUs).
- More memory and computation would enable a finer-grained blueprint that would lead to better performance but would also result in Pluribus using more memory or being slower during real-time search.
- set the size of the blueprint strategy abstraction to allow Pluribus to run during live play on a machine with no more than 128 GB of memory while storing a compressed form of the blueprint strategy in memory.

● **Abstraction for large imperfect-information games**

- To reduce the complexity of the game, we eliminate some actions from consideration and also bucket similar decision points together in a process called abstraction (24, 25). After abstraction, the bucketed decision points are treated as identical. We use two kinds of abstraction in Pluribus: action abstraction and information abstraction.

■ Action abstraction:

- reduces the number of different actions the AI needs to consider.
- To reduce the complexity of forming a strategy, Pluribus only considers a few different bet sizes at any given decision point.
- The exact number of bets it considers varies between 1 and 14 depending on the situation. Although Pluribus can limit itself to only betting one of a

few different sizes between \$100 and \$10,000, when actually playing no-limit poker, **the opponents are not constrained to those few options.**

- What happens if an opponent bets \$150 while Pluribus has only been trained to consider bets of \$100 or \$200? Generally, Pluribus will rely on its search algorithm (described in a later section) to compute a response in real-time to such “off-tree” actions.

■ Information abstraction:

- decision points that are similar in terms of what information has been revealed (in poker, the player’s cards and revealed board cards) are bucketed together and treated identically
- For example, a 10-high straight and a 9-high straight are distinct hands but are nevertheless strategically similar. Pluribus may bucket these hands together and treat them identically, thereby reducing the number of distinct situations for which it needs to determine a strategy.
- drastically reduces the complexity of the game, but it may wash away subtle differences that are important for superhuman performance.
- Therefore, during actual play against humans, Pluribus uses information abstraction only to reason about situations on future betting rounds, never the betting round that it is actually in. Information abstraction is also applied during offline self-play

● Montecarlo simulation:

- a computational algorithm that uses random sampling to solve problems. It's used to approximate functions and simulate the behavior of complex systems.
- The Monte Carlo simulation is a mathematical technique that predicts possible outcomes of an uncertain event.
 - Computer programs use this method to analyze past data and predict a range of future outcomes based on a choice of action.
 - For example, if you want to estimate the first month’s sales of a new product, you can give the Monte Carlo simulation program your historical sales data.
 - The program will estimate different sales values based on factors such as general market conditions, product price, and advertising budget.
- The Monte Carlo simulation is a probabilistic model that can include an element of uncertainty or randomness in its prediction. When you use a probabilistic model to simulate an outcome, you will get different results each time. **For example, the distance between your home and office is fixed. However, a probabilistic simulation might predict different travel times by considering factors such as congestion, bad weather, and vehicle breakdowns.**
- In contrast, conventional forecasting methods are more deterministic. They provide a definite answer to the prediction and cannot factor in uncertainty. For instance, they might tell you the minimum and maximum travel time, but both answers are less accurate.

AI now masters six-player poker

Abstract

Theoretical and practical challenges of multiplayer games

Description of Pluribus

Experimental evaluation

Conclusions

Acknowledgments

Supplementary Material

References and Notes

eLetters (2)

The blueprint strategy in Pluribus was computed using a variant of counterfactual regret minimization (CFR) (29). CFR is an iterative self-play algorithm in which the AI starts by playing completely at random but gradually improves by learning to beat earlier versions of itself. Every competitive Texas hold'em AI for at least the past 6 years has computed its strategy using some variant of CFR (4–6, 23, 28, 30–34). We use a form of Monte Carlo CFR (MCCFR) that samples actions in the game tree rather than traversing the entire game tree on each iteration (33, 35–37).

On each iteration of the algorithm, MCCFR designates one player as the traverser whose current strategy is updated on the iteration. At the start of the iteration, MCCFR simulates a hand of poker based on the current strategy of all players (which is initially completely random). Once the simulated hand is completed, the AI reviews each decision that was made by the traverser and investigates how much better or worse it would have done by choosing the other available actions instead. Next, the AI reviews each hypothetical decision that would have been made following those other available actions and investigates how much better it would have done by choosing the other available actions, and so on. This traversal of the game tree is illustrated in Fig. 2. Exploring other hypothetical outcomes is possible because the AI knows each player's strategy for the iteration and can therefore simulate what would have happened had some other action been chosen instead. This counterfactual reasoning is one of the features that distinguishes CFR from other self-play algorithms that have been deployed in domains such as Go (9), Dota 2 (20), and StarCraft 2 (21).

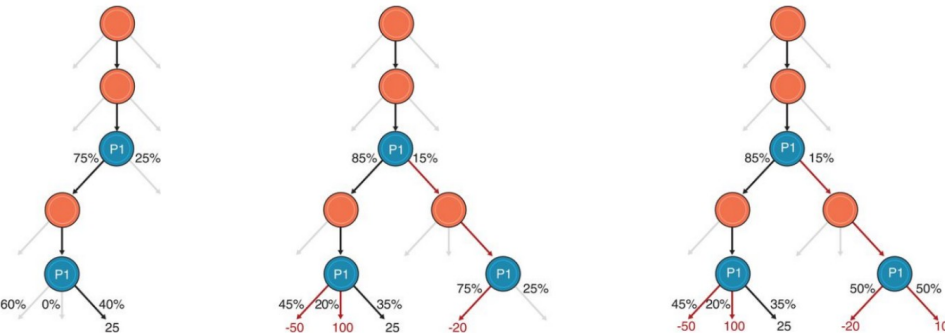


Fig. 2 A game tree traversal via Monte Carlo CFR. In this figure, player P_1 is traversing the game tree. **(Left)** A game is simulated until an outcome is reached. **(Middle)** For each P_1 decision point encountered in the simulation in the left panel, P_1 explores each other action that P_1 could have taken and plays out a simulation to the end of the game. P_1 then updates its strat-



Translating Pluribus-style multi-agent game-solving to a Mario Kart racing AI:

Mario Kart is a multi-agent game.

Poker ingredient	Mario Kart counterpart	Why it matters
Players (6 elite pros)	Karts (8 racers, human or AI)	Non-cooperative, each maximises finishing rank.
Hand + private	Current kart state (speed, drift angle,	Imperfect information—opponents'

cards	item inventory)	item boxes are hidden.
Common cards / board	Track, item boxes, hazards	Shared but stochastic; items spawn randomly.
Bet sizes	Action primitives (steer left/right, accelerate, brake, drift, use item)	Requires action <i>abstraction</i> to keep search tractable.
Pay-off / chips	Lap time, rank, coins	Dense reward shaping possible (e.g. +time gain, -collision).

Thinking about a Blueprint + real-time search → two-level racing brain

Phase	Pluribus technique (poker)	Mario Kart adaptation
Offline self-play to build a “blueprint”	Monte-Carlo CFR over an <i>abstracted</i> game tree	Train a multi-agent RL policy (PPO/MADDPG) in the Mario Kart simulator. Discretise the continuous controls into ≈ 10 –20 macro-actions (e.g. <i>hard-left-drift</i> + <i>full-throttle, fire-shell</i> + <i>hold-line</i>).
Runtime local improvement	Depth-limited search that re-solves the subgame reached at each decision point	Run a model-predictive game-theoretic planner each frame: 1. Sample opponents’ next k macro-actions using their policy. 2. Solve a dynamic game for an approximate Nash equilibrium trajectory (iterative best response). 3. Execute the first control and repeat.

Why split it? The offline blueprint gives global race-craft (corner set-ups, item economy) while the online solver reacts to “banana-split seconds” like sudden shells or overtakes.

Key engineering tricks (trying to mirror Pluribus)

Strategy Ideas	How to possibly use
Action abstraction	Cluster wheel-angle $\times$ throttle $\times$ drift combos into $\sim N$ “moves”. Items become extra discrete actions.
Information abstraction	Bucket track positions into sectors & lane-offset bins; bucket item inventories into threat levels (“has red shell”, “defence item”, “empty”).

Monte-Carlo rollouts	When an off-tree event occurs (e.g. opponent uses Lightning you never trained on), fall back to Monte-Carlo sampling to estimate Q-values for novel states.
Self-play scalability	Use domain-randomised emulation (different tracks, karts, physics noise) so the blueprint generalises—NeuralKart showed an emulator can run fast enough for massive data .
Computational budget	Pluribus used ~12 k CPU-hours ; a similar budget buys hundreds of millions of simulated Kart frames

<https://cs231n.stanford.edu/reports/2017/pdfs/624.pdf>

Can try:

Using offline self play MARL to learn a coarse “blueprint” policy, then refine it in real time with a fast Nash-equilibrium trajectory solver that reasons about rivals’ reactions—exactly the Pluribus two-phase recipe, but with karts instead of cards.

Mario Kart Research Direction

Pluribus to Pokemon Battles

Translating Pluribus to Pokemon Gen 1 Random Battles

- Pluribus uses Iterative Monte Carlo CFR (MCCFR)
 - variant of CFR that uses sampling to approximate the regret updates, making it more efficient for large game trees
- **Similarities:**
 - They both have imperfect information
- **Differences:**
 - Must adapt the algorithm to handle the sequential nature of Pokémon battles. In poker, everyone gets an individual independent move. In Pokemon, moves are sequential (both players make a choice, and that’s it)

- Unlike poker, Pokemon battles have visible states, simplifying some aspects of decision-making.
- Must represent each game state as a node in the game tree.

Steps:

- Use random sampling to simulate different battle outcomes and then update regrets based on sampled outcomes.
- Run multiple iterations
- Although Libratus is meant to be played in 1v1 games

CFR Web Simulation Notes

```
(base) mauriciogarcia@Mauricios-MacBook-Pro cfr_web % python game_loop.py
```

After 1000 games, win rate = 0.509

After 2000 games, win rate = 0.502

After 3000 games, win rate = 0.508

After 4000 games, win rate = 0.507

After 5000 games, win rate = 0.507

After 6000 games, win rate = 0.507

After 7000 games, win rate = 0.507

After 8000 games, win rate = 0.507

After 9000 games, win rate = 0.508

After 10000 games, win rate = 0.507

Final P1 win rate over 10000 games: 0.507

```
(base) mauriciogarcia@Mauricios-MacBook-Pro cfr_web %
```

^Shows equilibrium was found